# Continuous Integration, Delivery, Deployment

Лекция 11: CI/CD

Материалы (контент) могут использоваться исключительно в учебных и информационных целях без цели извлечения прибыли с обязательным указанием следующей информации:
© ООО «КНС ГРУПП» (YADRO), 2022-2023, все права защищены.
Авторы – Александрова О. В., Андрианов А.А., Денисенко Б. А., Дутова А. А., Ефимов М. А., Клочкова Л. И., Королев Д. О., Котлярова Л. П., Никифоров И. В., Ромашов В. А., Тянутов М. В.

# Определение

CI/CD - это комбинация непрерывной интеграции (Continuous Integration) и непрерывного развертывания (Continuous Delivery или Deployment). Это методология разработки программного обеспечения, которая позволяет автоматизировать процесс сборки, тестирования и развертывания приложений.

CI/CD позволяет разработчикам быстро и безопасно внедрять изменения в коде и ускорять процесс разработки.

# Continuous Integration

Непрерывная интеграция (Continuous Integration, CI) - это практика, которая предполагает, что все изменения кода интегрируются в общий кодовую базу на регулярной основе. Когда разработчик завершает работу над новой функцией или исправлением ошибки, он отправляет свой код в центральный репозиторий.

Этот процесс позволяет обнаруживать ошибки и конфликты с другими частями кода раньше, чем они станут серьезной проблемой. Также это позволяет ускорить процесс разработки, так как разработчики не будут ждать, пока другие закончат свою работу, чтобы интегрировать свой код.

# Цикл Continuous Integration

На этом этапе выполняются следующие действия:

- Получение последней версии кода из репозитория.
- Сборка приложения.
- Запуск автоматических тестов для проверки правильности работы приложения.
- Создание отчета о результатах тестирования.

# Continuous Delivery

Непрерывная доставка (Continuous Delivery, CD) - это практика, которая предполагает автоматизацию процесса доставки приложения в производственную среду. Она предполагает, что после каждого изменения кода приложение должно проходить автоматический процесс сборки, тестирования и развертывания в тестовой среде.

Данная практика позволяет быстро проверять, работает ли новый код и не содержит ли он ошибок. Если процесс проверки успешен, новый код готов к развертыванию в производственной среде.

# Цикл Continuous Delivery

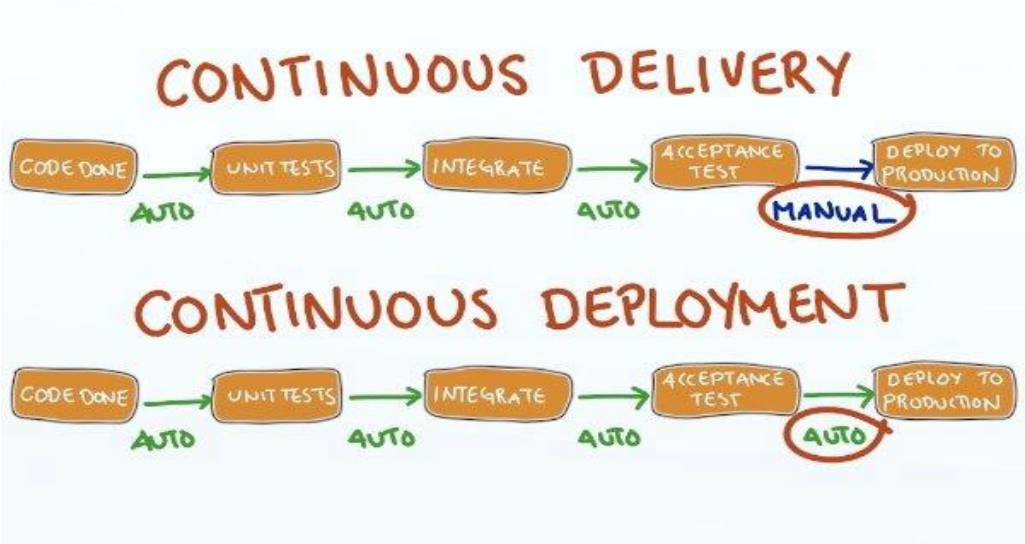
На этом этапе выполняются следующие действия:

- Создание нового релиза приложения на основе успешной сборки и прохождения всех тестов.
- Автоматическое развертывание нового релиза в тестовой среде.
- Запуск автоматических тестов в тестовой среде для проверки правильности работы приложения.
- Создание отчета о результатах тестирования.

# Continuous Deployment

**Непрерывное развертывание (Continuous Deployment, CD)** - это процесс автоматического развертывания приложения в производственной среде после успешного прохождения тестирования в тестовой среде в рамках CI/CD.

В отличие от непрерывной доставки (Continuous Delivery, CD), где развертывание происходит вручную, при непрерывном развертывании приложение разворачивается автоматически, без участия человека. Это позволяет ускорить процесс развертывания и уменьшить вероятность ошибок, связанных с ручным вмешательством.



# Цикл Continuous Deployment

На этом этапе выполняются следующие действия:

- Создание нового релиза приложения на основе успешной сборки и прохождения всех тестов.
- Автоматическое развертывание нового релиза в тестовой среде.
- Запуск автоматических тестов в тестовой среде для проверки правильности работы приложения.
- Автоматическое развертывание нового релиза в производственной среде.
- Проверка работоспособности приложения в производственной среде.
- Создание отчета о результатах проверки.

# Основные принципы CI/CD

- Частое коммитирование кода в общий репозиторий
- Автоматическая проверка кода на ошибки и соответствие стандартам качества
- Автоматическое создание артефактов (собранных версий программы) из проверенного кода
- Автоматическое развертывание артефактов в различных средах (тестовая, стейджинг, продакшн)
- Автоматическое получение обратной связи о работе программы в разных средах

# Преимущества CI/CD

- Ускорение процесса разработки и доставки программного обеспечения.
- Повышение качества и надежности программного обеспечения.
- Снижение рисков ошибок и сбоев.
- Улучшение коммуникации и сотрудничества в команде.

# Реализация CI/CD

Для реализации CI/CD необходимо использовать специальные инструменты, которые называются CI/CD серверами или платформами. Это программы, которые запускаются на отдельных серверах или облачных сервисах и выполняют задачи по автоматизации процесса разработки.

Примеры таких инструментов:

- Jenkins
- GitLab CI/CD
- GitHub Actions
- Travis CI

# Реализация CI/CD

CI/CD серверы работают по следующему алгоритму:

1. Отслеживают изменения в коде в общем репозитории (например, GitHub или GitLab)
2. Запускают задачи по сборке, тестированию и анализу кода (юнит-тесты, линтеры, статический анализ и т.д.)
3. Создают артефакты из проверенного кода (исполняемые файлы, контейнеры, пакеты и т.д.)
4. Развертывают артефакты в разных средах (на локальных или удаленных серверах, виртуальных машинах или контейнерах)
5. Собирают и отображают результаты работы программы в разных средах (логи, метрики, отчеты о тестировании и т.д.)

# Jenkins

Основные особенности Jenkins:

- Open-source
- Написан на Java
- Большое сообщество и плагины
- Поддержка множества языков и технологий
- Гибкость и настраиваемость
- Интеграция с другими инструментами DevOps
- Масштабируемость
- Поддержка различных ОС

# Jenkins

Ключевые возможности Jenkins:

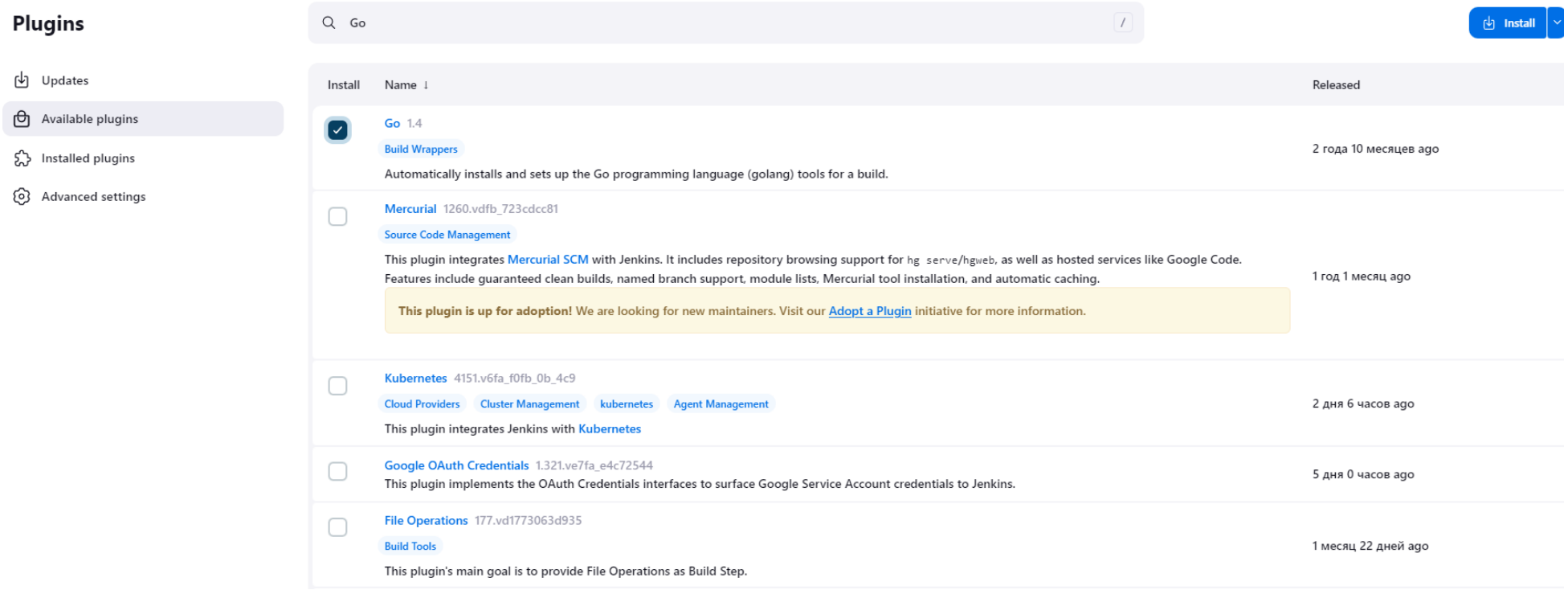
- Создание и управление множеством сборочных пайплайнов
- Интеграция с различными инструментами DevOps (Git, Docker, Ansible, Jira и т.д.)
- Скорость и масштабируемость
- Гибкость конфигурации
- Система уведомлений

# Работа с Jenkins. Создание CI pipeline

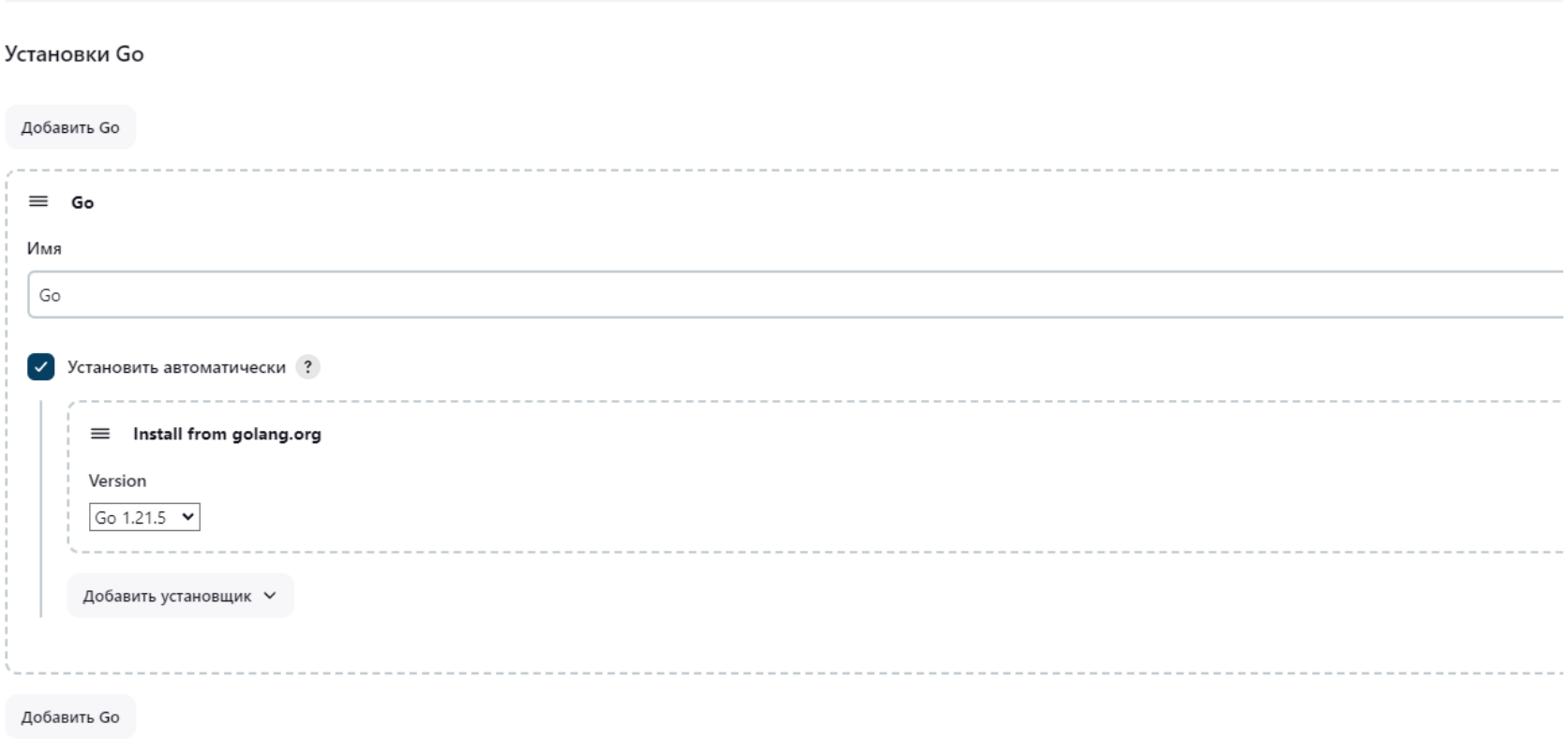
- Jenkins может взаимодействовать с Git (с помощью плагина)
- Jenkins может воспроизводить действия из файлов, расположенный в git репозиториях.
- Для описания CI pipeline используется Jenkinsfile, в котором описан порядок действий
- В Jenkins можно запускать различные инструменты (если установлен соответсвующий плагин) внутри pipeline.

# Работа с Jenkins. Создание CI pipeline

Для начала необходимо установить необходимые плагины для работы нужных инструментов, например языка Go. Для этого перходим во вкладку Настройки Jenkins => Plugins, где находим и устанавливаем необходимый плагин.



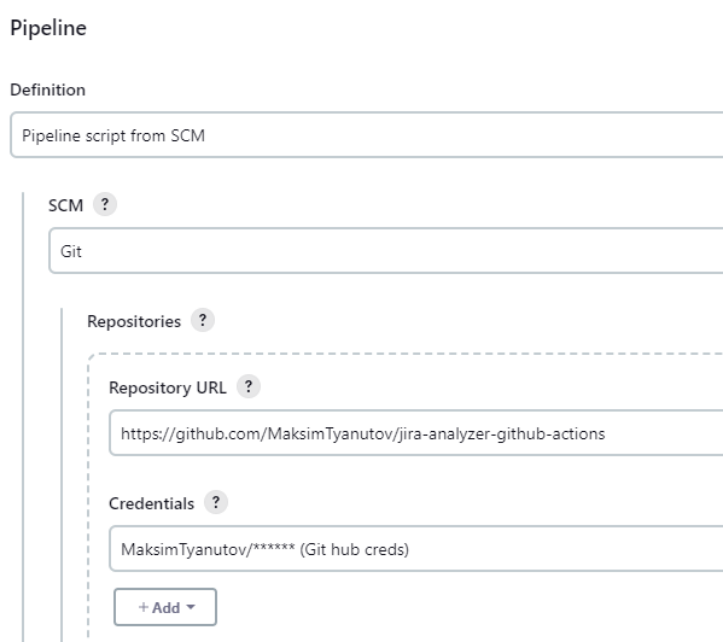
# Работа с Jenkins. Создание CI pipeline

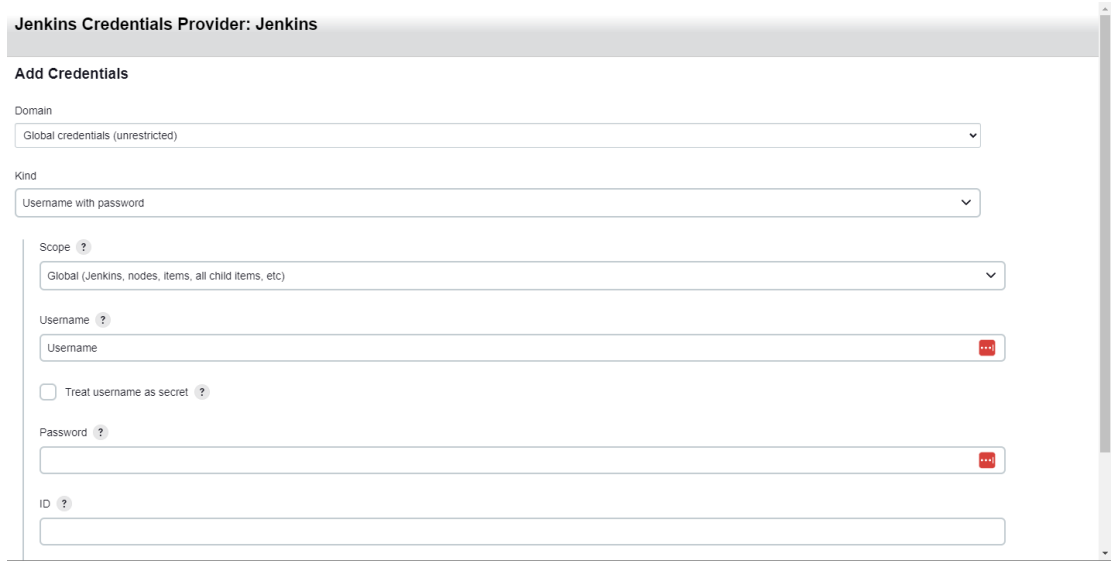


Во вкладке «Настройки Jenkins => Tools» происходит настройка инструментов пайплайнов, например компилятора Go. Для этого находим в списке Go и нажимаем кнопку добавить. Заполняем поле имени и версию. По этому имени мы будем обращаться к инструменту в коде.

# Работа с Jenkins. Создание CI pipeline

Добавление нового пайплайна – «Создать Item => Pipeline». В разделе «Pipeline => Definiton» выбираем Pipeline script from SCM. В качестве SCM указывается Git. В качестве Credentials используется Github username и personal access token  как пароль.



# Работа с Jenkins. Создание CI pipeline

Сам сценарий пайплайна хранится в файле Jenkinsfile в репозитории. Пример содержания такого файла:

```
pipeline {
    agent any

    tools {
        // Указываем инструмент Go
        go 'Go'
        // Указываем инструмент Node.js
        nodejs 'NodeJS'
    }

    stages {

...
```

Jenkinsfile

Начало пайплайна

Указание агентов, на которых этот пайплайн может запускаться

Интеграция инструментов Jenkins

Основной код сценария

# Работа с Jenkins. Создание CI pipeline

```
stages {
     stage('Go Tests') {
                 steps {
                     script {
                         // Переходим в директорию 'connectorJIRA' и
     выполняем тесты
                         dir('connectorJIRA') {
                             sh 'go test -v ./...'
                         }
                         // Переходим в директорию 'REST_API' и
     выполняем тесты
                         dir('REST_API') {
                             sh 'go test -v ./...'
                         }
                     }
                 }
             }

             stage('Web Tests') {
                 steps {
                     script {
                         // Переходим в директорию 'web'
                         dir('web') {
                             // Устанавливаем зависимости Node.js
                             sh 'npm install'
                             // Собираем веб-приложение
                             sh 'npm run build'
                         }
                     }
                 }
             }
     }
```

Начало отдельной стадии

Команды, реализующие сценарий пайплайна

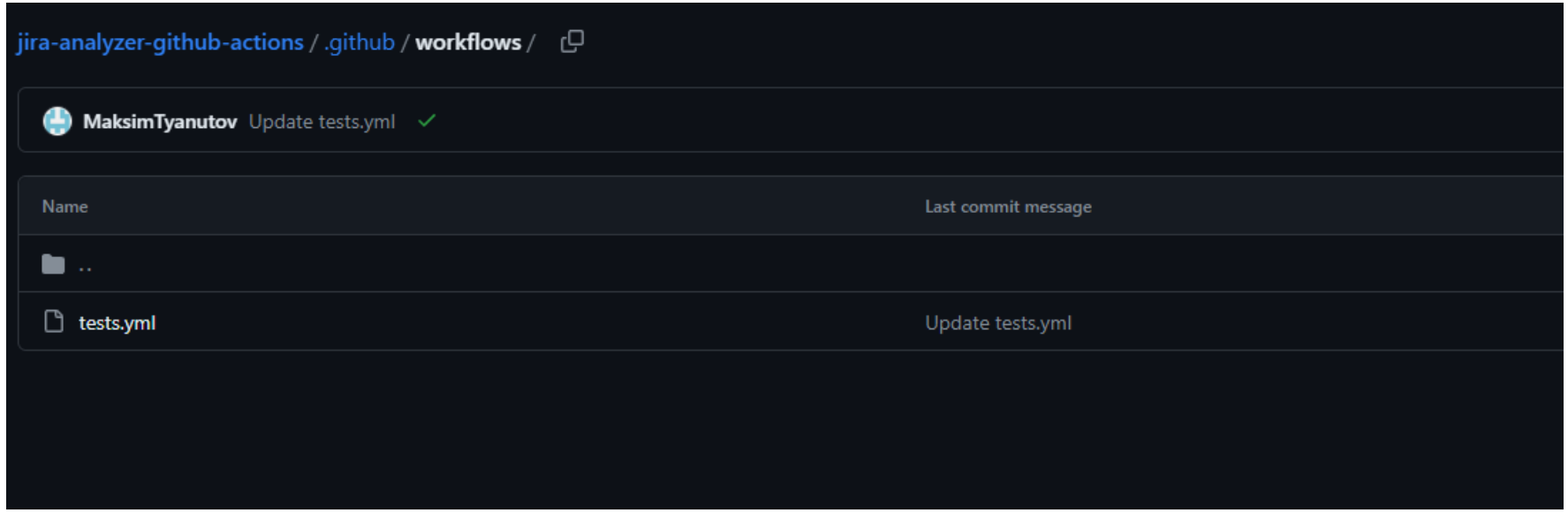
Jenkinsfile

# Пример Github Actions

GitHub Actions - это сервис, который позволяет автоматизировать различные задачи, связанные с разработкой программного обеспечения, такие как сборка, тестирование, доставка и развертывание кода. GitHub Actions интегрирован с GitHub и доступен бесплатно ограниченное количество минут в месяц.

# Пример Github Actions

GitHub Actions работает на основе файлов конфигурации, называемых workflow, которые описывают последовательность шагов (actions), которые нужно выполнить при наступлении определенных событий (events) в репозитории.



# Пример Github Actions

Пример файла конфигурации tests.yml:

```yaml
name: Tests

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:

  Go-Tests:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v3

    - name: Set up Go
      uses: actions/setup-go@v3
      with:
        go-version: 1.19

...
```

Tests.yml

Название workflow

События, при которых workflow будет запускаться

Задания (jobs), которые будут выполняться в рамках данного workflow

Основной код сценария workflow

# Пример Github Actions

Пример файла конфигурации tests.yml:

```yaml
jobs:

  Go-Tests:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v3

    - name: Set up Go
      uses: actions/setup-go@v3
      with:
        go-version: 1.19

    - name: connector-jira
      working-directory: ./connectorJIRA
      run: go test -v ./...

    - name: backend
      working-directory: ./REST_API
      run: go test -v ./...

  Web-tests:
    runs-on: ubuntu-latest
...
```

Название задания. Каждое задание выполняется независимо от других

Docker контейнер, на котором будет исполняться задание

Параметр **uses** используется для выбора действия (**action**), которое уже определено и может быть повторно использовано. В данном случае данная команда переключает репозиторий на временную ветку

Установка **go** с помощью **uses** и установки параметра с помощью with

Исполнение команды с помощью **run** – запуск тестов

Tests.yml

# Пример Github Actions

Теперь при коммите в ветку main у нас будут автоматически запускаться тесты.

**jira-connector**

succeeded 23 minutes ago in 21s

- Set up job
- Run actions/checkout@v3
- Set up Go
- Tests
- Post Set up Go
- Post Run actions/checkout@v3
- Complete job

Tests

```
1  ▶ Run go test -v ./...
4  go: downloading github.com/sirupsen/logrus v1.9.0
5  go: downloading golang.org/x/sync v0.1.0
6  go: downloading github.com/andygrunwald/go-jira v1.16.0
7  go: downloading github.com/lib/pq v1.10.7
8  go: downloading gopkg.in/yaml.v3 v3.0.1
9  go: downloading golang.org/x/sys v0.1.0
10 go: downloading github.com/fatih/structs v1.1.0
11 go: downloading github.com/golang-jwt/jwt/v4 v4.4.2
12 go: downloading github.com/google/go-querystring v1.1.0
13 go: downloading github.com/pkg/errors v0.9.1
14 go: downloading github.com/trivago/tgo v1.0.7
15 ?       connectorJIRA/cmd       [no test files]
16 ?       connectorJIRA/pkg/apiserver     [no test files]
17 ?       connectorJIRA/pkg/connector     [no test files]
18 ?       connectorJIRA/pkg/datapusher    [no test files]
19 === RUN   TestFormatIssues
20 === RUN   TestFormatIssues/HADOOP-400_correct_parsing
21 === RUN   TestFormatIssues/HADOOP-400_incorrect_unmarshal
```

yadro.com